

```
D:\LISP\pathfinding>sbcl --load run.lisp
This is SBCL 2.3.2, an implementation of ANSI Common Lisp.
More information about SBCL is available at <http://www.sbcl.org/>.

SBCL is free software, provided as is, with absolutely no warranty.
It is mostly in the public domain; some portions are provided under
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
BFS path: (A C F)
DFS path: (A B E F)
A* path: (A C F)
```

Figure 2: A visual illustration of the BFS, DFS, and A* search routes

```
(progn
  (push node visited)
  (dolist (n (neighbors node graph))
    (unless (member n path)
      (push (append path (list n)) open))))))

(load "graph.lisp")           ; load *graph*. Position and
                              ; neighbour helpers
(load "bfs.lisp")             ; load bfs.lisp
(load "dfs.lisp")             ; load dfs.lisp
(load "astar.lisp")           ; load astar.lisp

(in-package :pathfinding)
(format t "BFS path: ~a~%" (bfs 'A 'F *example-graph*))
(format t "DFS path: ~a~%" (dfs 'A 'F *example-graph*))
(format t "A* Path: ~a~%" (a-star 'A 'F *example-graph*))
```

Implementation

The SBCL Common-Lisp platform is used to execute these codes. As SBCL is a free, open source program, it is downloadable and can be installed on a Windows-based PC. It is better to have a project directory 'pathfinding' for this implementation, as mentioned below, with six individual Lisp files.

```
pathfinding/
  — package.lisp           ; package declaration
— graph.lisp               ; defines *graph* position and
neighbors
— bfs.lisp                 ; define Breadth-First Search
— dfs.lisp                 ; define Depth-First Search
— astar.lisp               ; define A* search
— run.lisp                 ; loads everything and runs
```

The *project.lisp* defines a project under which all the defined Lisp codes are integrated and can be executed using the *run.lisp*. The *run.lisp* loads all the necessary files for the execution. Since the project name is *pathfinding*, all the loaded Lisp files should have (*in-package: pathfinding*) at the top of the file. The package file is:

```
;; package.lisp
defpackage :pathfinding
  (:use :cl)
  (:export :bfs :dfs :a-star :example-graph :example-
positions))
(in-package :pathfinding)
```

The run file is:

```
;; run.lisp
(load "package.lisp")
```

```
(load "graph.lisp")           ; load *graph*. Position and
                              ; neighbour helpers
(load "bfs.lisp")             ; load bfs.lisp
(load "dfs.lisp")             ; load dfs.lisp
(load "astar.lisp")           ; load astar.lisp

(in-package :pathfinding)
(format t "BFS path: ~a~%" (bfs 'A 'F *example-graph*))
(format t "DFS path: ~a~%" (dfs 'A 'F *example-graph*))
(format t "A* Path: ~a~%" (a-star 'A 'F *example-graph*))

The search path from the start node A to the goal node F
can be achieved by executing the run.lisp file with sbcl --load
run.lisp
```

Execution

Command line execution of *sbcl --load run.lisp* will produce a result as shown in Figure 2.

Paths of BFS, DFS, and A* to reach the goal F from the start node A are as follows:

```
BFS path: (A C F)
DFS path: (A B E F)
A* path: (A C F)
```

Visualisation

Here is how these three search strategies reach the goal node 'F'. As can be seen from the graphs in Figure 2, BFS is an exhaustive search, while DFS and the A* search (with Euclidean distance) routes are identical with our six-node linked network. Colour codes are used to indicate search paths: orange for A* searches, pink for DFS searches, and green for BFS searches.

You can try and experiment with various heuristic distance metrics and graphs. **END** 🐼

👤 By: Dr Dipankar Ray

The author has more than 30 years of experience in software development. He has published two books on MATLAB and medical image processing (published by PHI Learning and SPD).